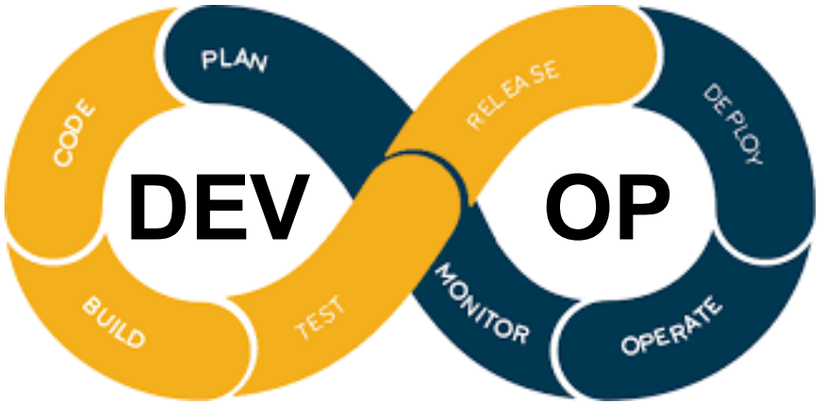
UNIVERSITAT DE BARCELONA

Continuous…



Design
Testing
Integration
Delivery/Deployment

by Eloi Puertas

What is **Continuous Design**

- Create and Modify the design of a system as it is developed.
- Continuous Design and Development implies:
- **TDD** (Test Driven Development)
- **Refactoring**

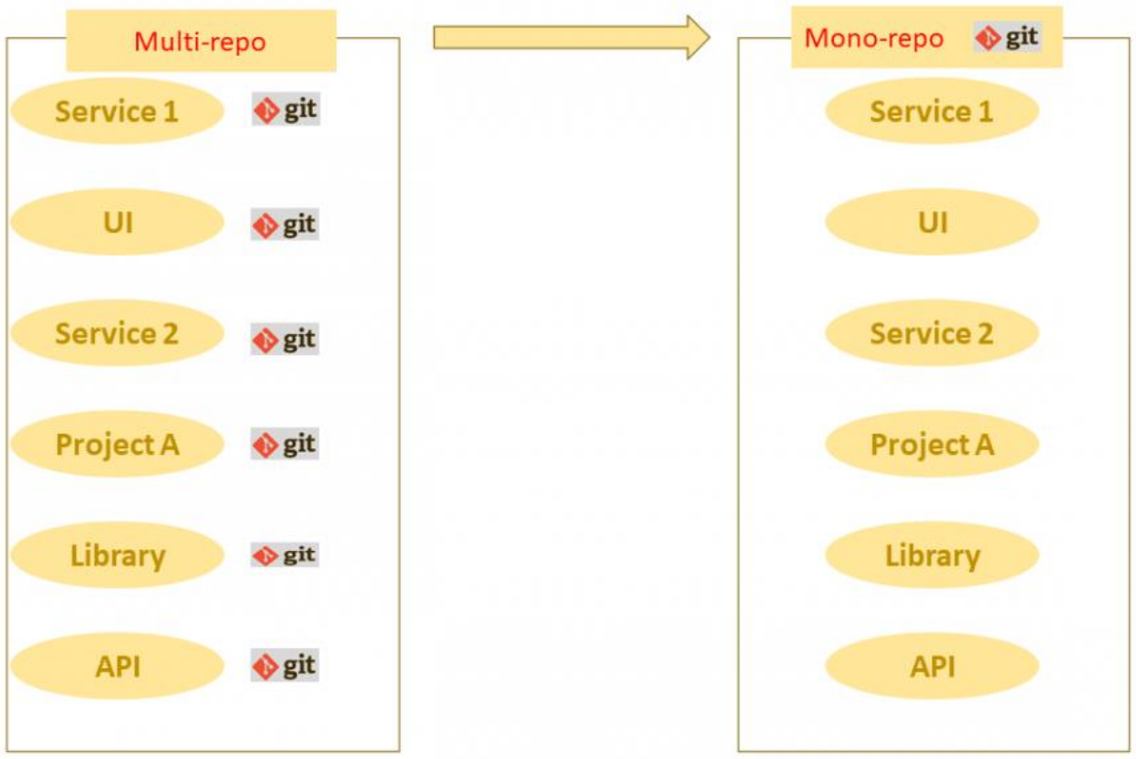
What is **Continuous Testing**

- Process of executing **automated tests** as part of the Agile sofware development process. (Self-Testing Code)
- Provides fast and continuous feedback
- Continuous Testing includes the validation of both
- **functional requirements:** unit Test, API testing, Integration Testing, System Testing…
- **non-functional requirements:** static code analysis, security testing, performance testing…

Repo Strategy

Mono-repo. All project live in one repo. favors consistency

Multi-repo. One repository per service or library. Focuses on decoupling. Ideal for microservices



Mono-Repo

+ A single place to store all the project code, and can be accessed by everyone on the team
+ Easy to reuse and share code, collaborate with the team
+ Easy to understand the impact of your change on the entire project
+ Best option for code-refactoring and large changes to code
+ Team members can get an overall view of the entire project
+ Easy to manage dependencies
- Performance: operations might become slow

Multi-Repo

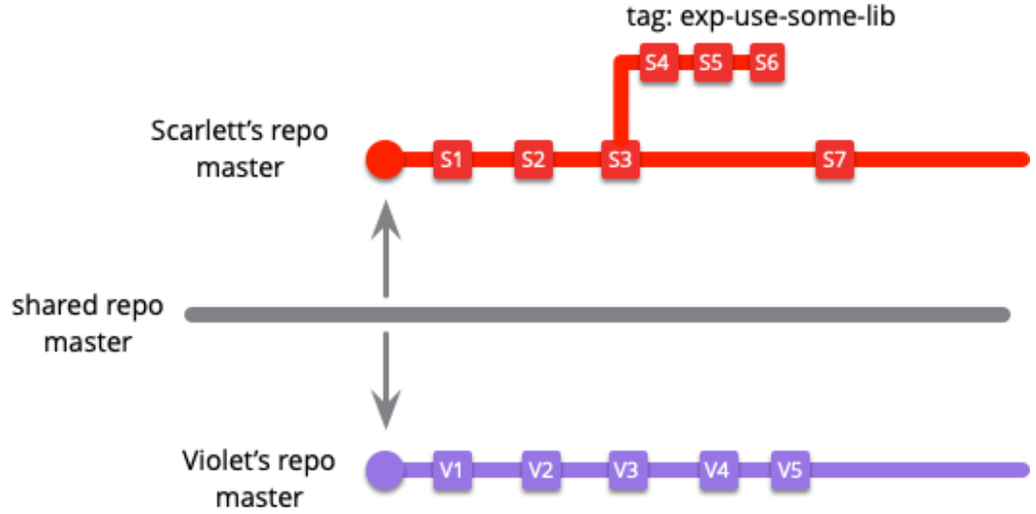
+ Each service and library have its own versioning
+ Code check-outs and pulls are small and separate, thus there are no performance issues even if the project size grows
+ Teams can work independently and need not have access to the entire codebase
+ Faster development and flexibility
+ Each service can be released separately and have its deployment cycle, thus making CI and CD easier to implement
+ Better access control. All teams don't need to have full access to all the libraries, but they can get read access if they need
- The dependencies and libraries used across services and projects have to be regularly synced to get the latest version
- Encourages a siloed culture at some point, leading to duplicate code and individual teams trying to resolve the same problem
- Each team may follow a different set of best practices for their code causing difficulties in following common best practices
- Running end-to-end tests can be hard

meta: A tool for managing multi-project

Branching and Code Integration

- *Source Branching:* Create a copy and record all changes to that copy. Branch per person, to avoid conflicts.
- *Mainline:* A single, shared, branch that acts as the current state of the product
- *Healthy Branch:* On each commit, perform automated checks, usually building and running tests, to ensure there are no defects on the branch

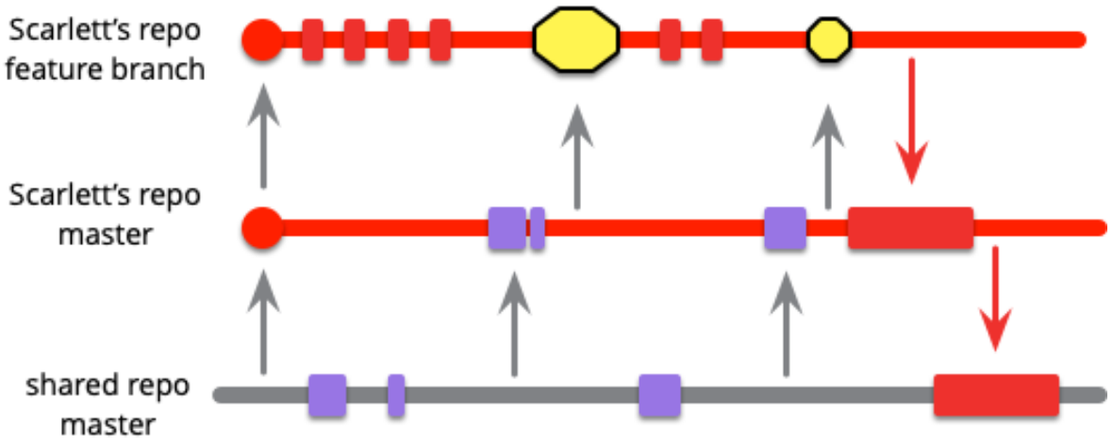
branching is easy, merging is harder.



https://martinfowler.com/articles/branching-patterns.html#integration-patterns

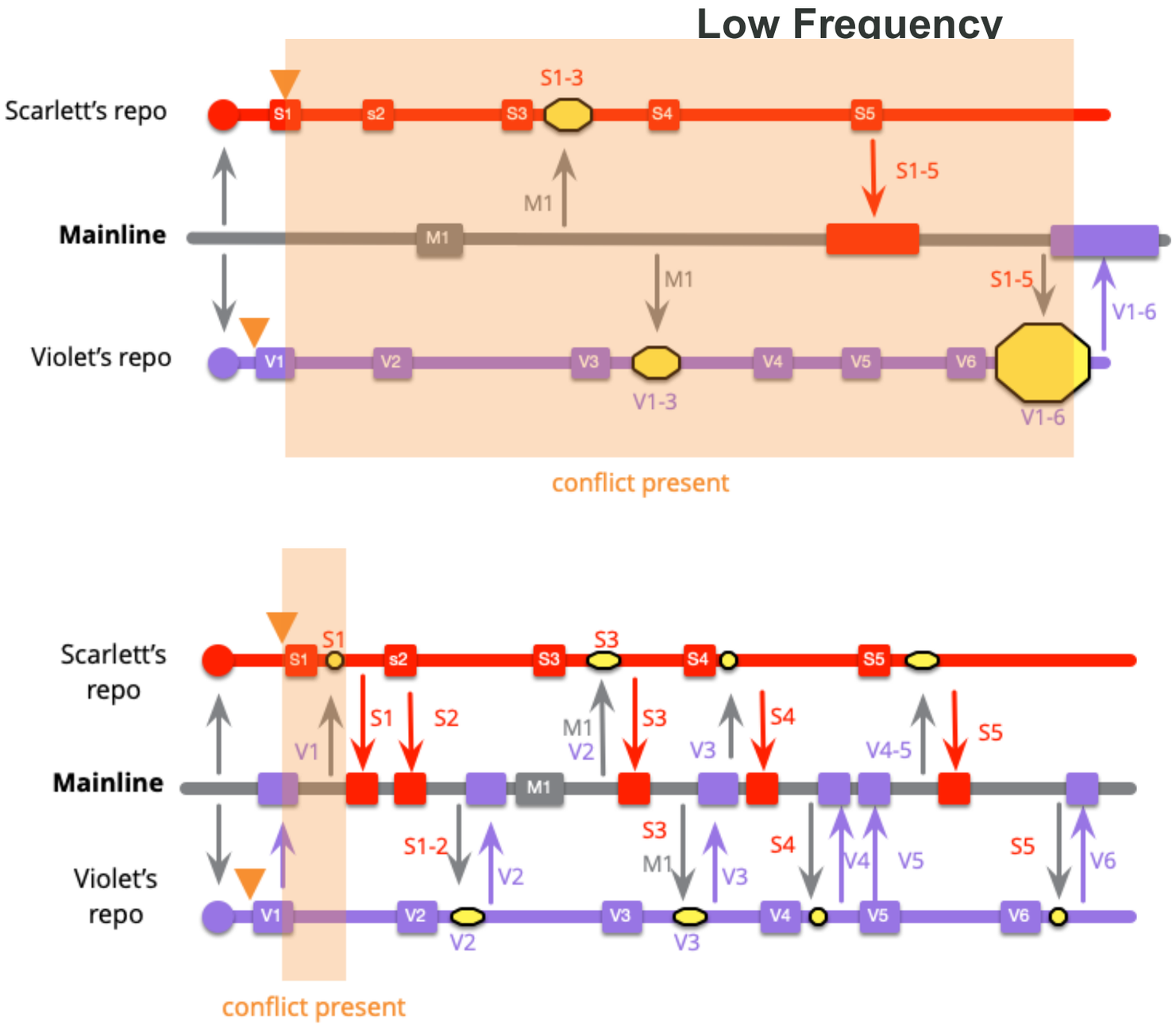
Feature Branching

- *Feature Branching*. Put all work for a feature on its branch, and integrate it into the mainline when the feature is complete.



https://martinfowler.com/articles/branching-patterns.html#integration-patterns

Integration Frequency



- Frequent integration increases the frequency of merges but reduces their complexity and risk.

https://martinfowler.com/articles/branching-patterns.html#integration-patterns

What is **Continuous Integration (CI)**

- Developers do mainline integration as soon as they have a healthy commit they can share, usually less than a day's work, reaching frequent integration points with a partially built feature.
- To hide partially built feature we can use *feature flags*
- The difference between **feature branching** and **continuous integration** isn't whether or not there's a feature branch, but when developers integrate with the mainline.

Feature Branching vs Continuous Integration

Feature Branching (pull request)

- All the code in a feature can be assessed for quality as a unit
- Feature code only added to product when feature is complete
- Less frequent merges

Continuous Integration (github actions)

- Supports higher frequency integration than feature length
- Reduced time to find conflicts
- Smaller merges
- Encourages refactoring
- Requires commitment to healthy branches (and thus self-testing code)
- Scientific evidence that it contributes to higher software delivery performance

Source: Martin Fowler, Patterns for Managing Source Code Branches (martinfowler.com/articles/branching-patterns.html)

Continuous Integration is a practice, not a tool

Continuous Build: a CI server that only builds feature branches on each commit.

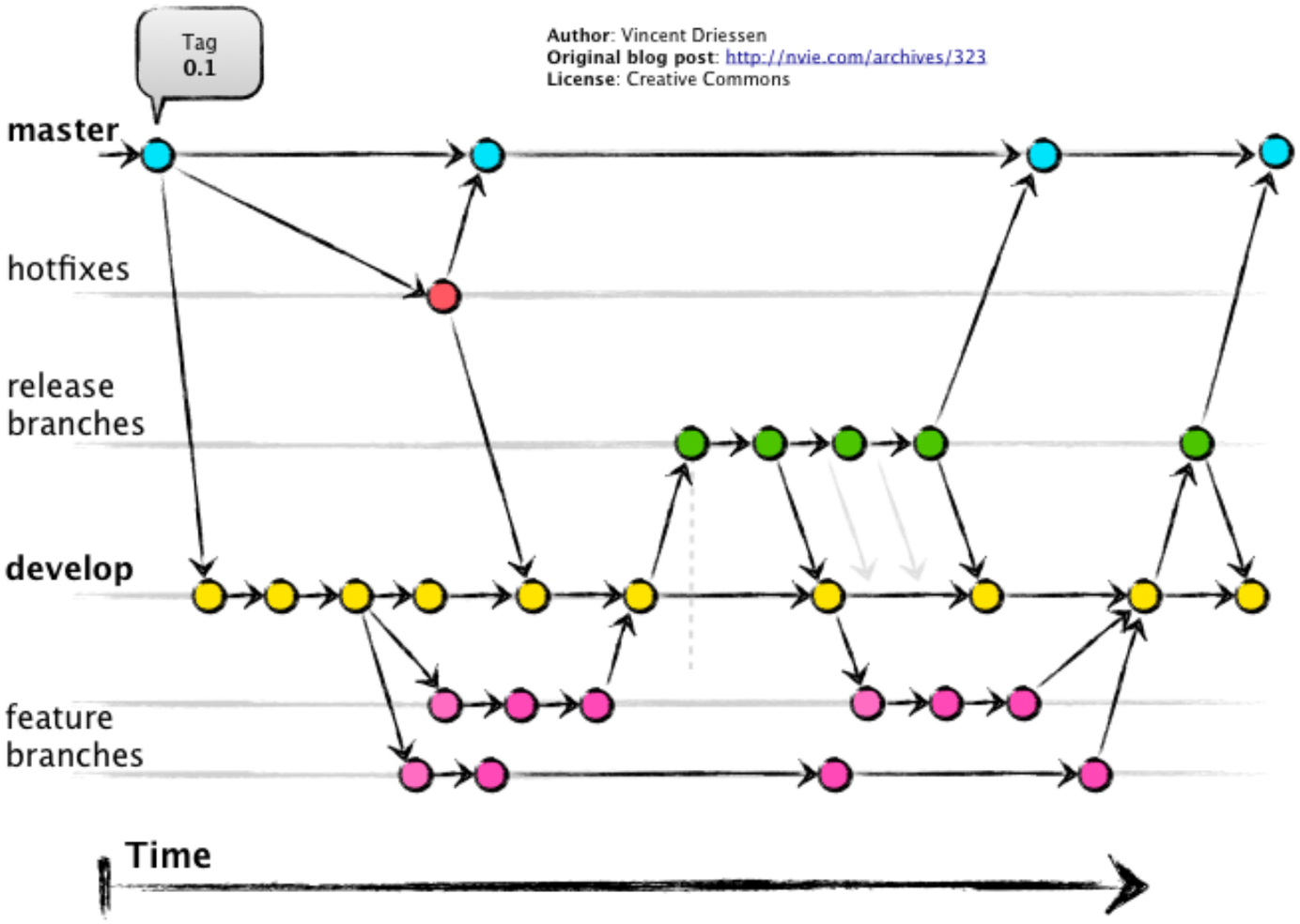
Continuous Integration: GitHub Actions is the tool; the practice is integrating healthy commits into the mainline at least daily.

Pull request: Feature Branching plus pre-integration review.

Source: Martin Fowler, Patterns for Managing Source Code Branches (martinfowler.com/articles/branching-patterns.html)

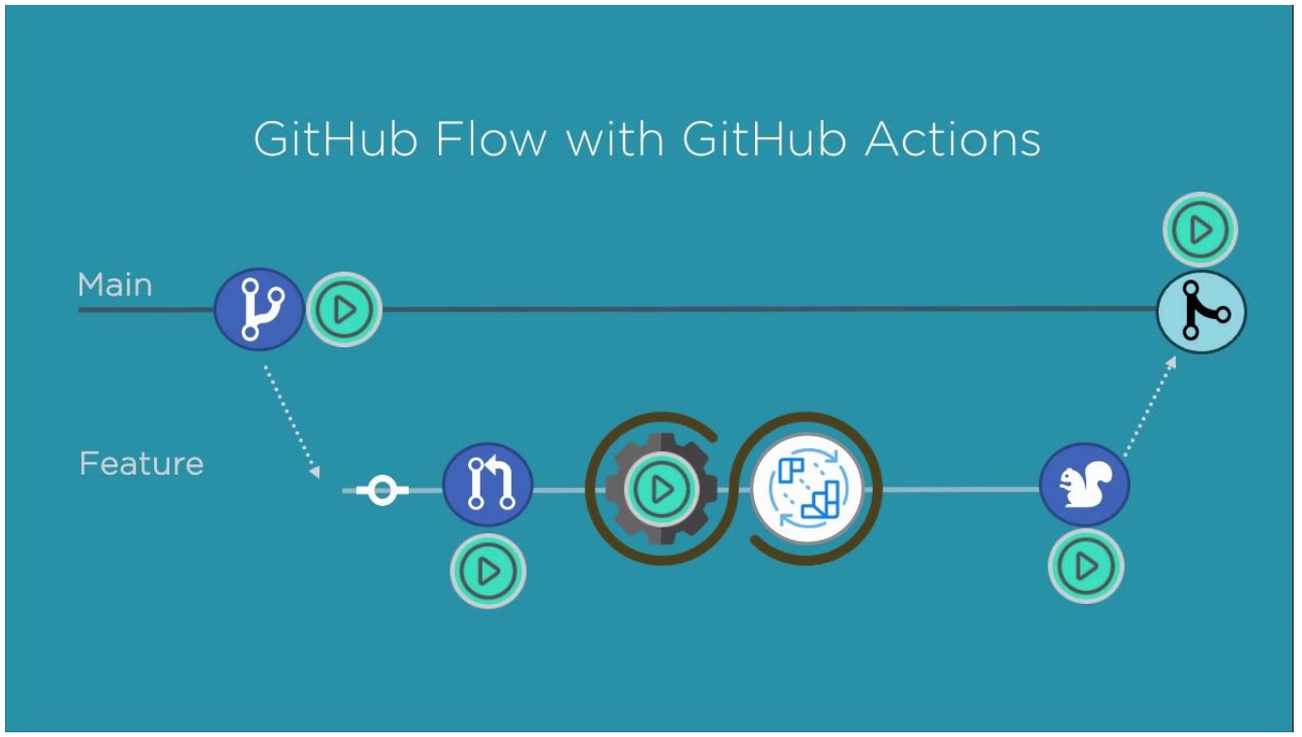
Branching Workflows and policies

GitFlow Workflow



Branching Workflows and policies

GitHub Workflow



Workflow:

- Run tests locally: run and pass all **unit tests**
- Compile code in CI: After every commit compile in the build server
- Run tests in CI: Run test units, integration tests, static analyses, profile performance… in the builder server
- Ready for Deploy an artifact from CI: the code in the builder server is ready for CD

GitFlow vs GitHub Flow

GitFlow

Long-lived develop and master branches, plus feature, release and hotfix branches.

Designed for versioned software with several releases in production.

Release branches mean the mainline is not always release-ready.

GitHub Flow

One mainline plus feature branches, merged through pull requests.

Assumes a single version in production and a release-ready mainline.

Production fixes follow the normal flow, with no hotfix branches.

In 2020 GitFlow's author advised teams doing continuous delivery to adopt a simpler workflow, such as GitHub Flow.

Sources: Vincent Driessen, A successful Git branching model (nvie.com/posts/a-successful-git-branching-model); Martin Fowler.

What is **Continuous Delivery/Deployment (CD)**

- **Continuous Delivery**: Makes sure the software checked in on the **“dev”** line is always in a state that can be deployed.

- **Continuous Deployment:** Makes the deployment process fully automated.

Deployment Strategies

Including a deployment stage before release that utilizes a realistic testing environment

- **Blue/green deployment:** Set up two identical environments, with only one going live at a given time. Initially roll out new releases to the offline environment, and, if successful, switch to the new environment, and the original production environment becomes idle. The first environment provides a backup, allowing you to switch back if there is an issue with the new release.

- **Canary Release:** Release a new update to a subset of users to test it in a limited, real-world setting. If successful, you can roll out the deployment to a wider user base. If unsuccessful, you can roll back the release.

- **Deployment with A/B testing.** Deploys different versions of a feature to verify performance and usability. It is primarily used to review the effectiveness of a change and how the market reacts to the change.

- **Feature Flags.** Allows to enable or disable a feature during runtime, for some or all the users.

CI/CD Best Practices

- Automate the build
- Make the build self-testing
- Everyone integrates to the baseline every day
- Every merge (to baseline) should be built
- Every bug-fix commit should come with a test case
- Keep the build fast
- Test in a clone of the production environment
- Make it easy to get the latest deliverables
- Everyone can see the results of the latest build
- Implement Continuous Delivery first.
- Implement progressive delivery (feature flags)

Cloud Services for CI/CD

SaaS: Software as a Service. Web Application

PaaS: Platform as a Service. Web App for Deployment.

- (**dynamic website**): Heroku(**static website**): Netlify, Vercel.

IaaS: Infrastructure as a Service. Web App for Infrastructure Management.

- AWS, Azure, Google Cloud, DigitalOcean

CI/CD: CI: Github Actions / CD: Azure

GitHub Actions and Azure

CI/CD tutorials: concepts, continuous integration, deployment to Azure and board automation

Understanding GitHub Actions

CI/CD platform built into GitHub: it automates build, test and deployment.

Workflow: an automated process defined in YAML in **.github/workflows/**.

Event: what triggers it (push, pull_request, schedule, manual…).

Job: a set of steps on a fresh runner. Jobs run in parallel by default, or in sequence with **needs**.

Action: a reusable step (e.g. actions/checkout) from the Marketplace.

```yaml
name: Hello Actions
on: [push]
jobs:
  saluda:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hello ${{ github.sha }}"
```

Source: docs.github.com/en/actions/get-started/understand-github-actions

GitHub Actions for CI

CI: integrate often and verify every change automatically.

Build process: **compile, lint, test** on every push and pull_request.

The result shows on the PR as a **status check**; make it required with branch protection to keep a healthy branch.

strategy.matrix tests several versions in parallel.

cache: npm keeps the build fast.

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:
jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [20.x, 22.x]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run build --if-present
      - run: npm test
```

Sources: docs.github.com/en/actions/get-started/continuous-integration · …/tutorials/build-and-test-code/nodejs

GitHub Actions for CD to Azure

Deployment process: dockerize the app, publish the image to a registry and deploy it to Azure App Service.

Registry: Azure Container Registry (name.azurecr.io) or GitHub Container Registry (ghcr.io).

Azure authentication: publish profile, service principal or OpenID Connect, which Microsoft recommends.

Image tagged with github.sha: an immutable artifact, so rollback is trivial.

GitHub **environments** and Azure **deployment slots** for staging, approvals and blue/green.

Sources: learn.microsoft.com/en-us/azure/app-service/deploy-container-github-action · docs.github.com (Docker to Azure App Service)

CD to Azure: authentication with OIDC

```yaml
on:
  push:
    branches: [main]
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
```

CD to Azure: image and deployment

```
# inside steps, after the login
- run: |
    az acr login --name ${{ vars.ACR_NAME }}
    docker build -t $IMAGE .
    docker push $IMAGE
  env:
    IMAGE: ${{ vars.ACR_NAME }}.azurecr.io/app:${{ github.sha }}
- uses: azure/webapps-deploy@v2
  with:
    app-name: ${{ vars.WEBAPP_NAME }}
    images: ${{ vars.ACR_NAME }}.azurecr.io/app:${{ github.sha }}
```

With **required reviewers** on the production environment this is Continuous Delivery; without approval, Continuous Deployment.

Actions to manage the board

GitHub Projects has **built-in automations** with no YAML: for example, moving an item to Done when an issue or PR closes.

Built-in workflows can also **add items** that match a filter.

For custom flows: an Actions workflow plus the Projects GraphQL API, with a GitHub App token or a PAT.

actions/add-to-project adds new issues and PRs to the board.

```yaml
name: Add to board
on:
  issues:
    types: [opened]
  pull_request:
    types: [opened]
jobs:
  add:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/add-to-project@v1.0.2
        with:
          project-url: https://github.com/orgs/ORG/projects/N

          github-token: ${{ secrets.ADD_TO_PROJECT_PAT }}

          labeled: bug, enhancement
```

Sources: docs.github.com (Projects: built-in automations, automating projects using Actions) · github.com/actions/add-to-project

Summary: from commit to production

push/PR, CI (lint, build, test), merge to main, Docker image (SHA), registry, staging (slot), optional approval, production.

Tutorial	Goal	Typical trigger	Key actions	Result
Understanding GitHub Actions	Core concepts	Any event	actions/checkout	Workflow, job, step, runner
CI	Verify every change	push, pull_request	setup-node, matrix, cache	Green status check = healthy branch
CD to Azure	Deploy the artifact	push to main	azure/login, webapps-deploy, Docker	App in production or staging
Board flows	Project management	issues, pull_request	add-to-project, GraphQL	Board in sync with the work

Best practices with Actions

Branch protection and required status checks: main always green.

Pin actions to a major version or a SHA, and review them periodically.

Keep secrets in Settings, Secrets; prefer OIDC over static credentials.

concurrency prevents two simultaneous deployments.

Goal: CI under 10 minutes (keep the build fast).

Source: martinfowler.com/articles/continuousIntegration.html